

Discovering Hidden Relations Among Triangle Invariants

triangle-relations project

Abstract

We describe a numerical method for discovering previously unknown functional relations among scalar quantities derived from a triangle, in the spirit of Euler's classical relation between the circumradius, the inradius, and the distance between the incenter and circumcenter. Because a triangle has three degrees of freedom up to rigid motion, a relation among four or more derived scalars is guaranteed by dimension counting alone and is therefore uninformative; a relation among exactly three scalars, by contrast, is a genuine and generically unexpected algebraic coincidence. We detect such coincidences by training a bottleneck autoencoder on sampled triangle data and comparing its reconstruction error against a column-permuted null, then recover an explicit closed form for flagged candidates via a monomial null-space fit. Applied to the circumradius, inradius, and incenter-to-circumcenter distance, the method rediscovers Euler's relation end to end, with no prior knowledge of the formula.

Keywords: triangle geometry; Euler's relation; functional dependence; autoencoders; symbolic regression

1. Setup: the moduli space of triangles

A triangle is specified by three points in the plane: six real parameters. Every quantity we consider (area, perimeter, circumradius R , inradius r , distances between derived points, and so on) is built purely from the vertex coordinates and is invariant under rigid motions: translating or rotating the triangle changes none of them. Translations (2 parameters) and rotations (1 parameter) form a 3-dimensional group acting on the 6-dimensional space of vertex triples; quotienting it out leaves a 3-dimensional moduli space M of triangle shapes up to congruence (reflection is a further discrete symmetry and does not change this dimension count). Concretely, M can be coordinatized by the three side lengths (a, b, c) , subject to the triangle inequality.

Every derived scalar quantity is a smooth function $f : M \rightarrow \mathbb{R}$. Choosing k such quantities gives a smooth map $F = (f_1, \dots, f_k) : M \rightarrow \mathbb{R}^k$.

2. Why relations among four or more quantities are not interesting

If $k > \dim(M) = 3$, a generic smooth map from a 3-manifold into $\mathbb{R}^k$ has, as its image, an (at most) 3-dimensional submanifold of $\mathbb{R}^k$ -- embedding a 3-dimensional space into a higher-dimensional ambient space produces $k - 3$ constraint equations for free, regardless of which k functions were chosen, as long as they are not specially degenerate. This image is cut out locally by $k - 3$ independent equations $G_i(f_1, \dots, f_k) = 0$. So finding some relation among four or more triangle invariants is guaranteed by dimension counting alone -- it says nothing specific about triangle geometry.

3. Why a relation among exactly three quantities is special

If $k = \dim(M) = 3$ exactly, a generic map $F : M \rightarrow \mathbb{R}^3$ is, at a generic point, a local diffeomorphism: its Jacobian DF has full rank 3, and the triple (f_1, f_2, f_3) sweeps out a full 3-dimensional open region of $\mathbb{R}^3$ -- a solid blob, not a surface. An exact relation $G(f_1, f_2, f_3) = 0$ holding everywhere is equivalent to DF having rank at most 2 everywhere: the image collapses onto a 2-dimensional surface. That is not generic -- it is a genuine algebraic coincidence, exactly the kind of fact that constitutes a classical theorem. Euler's relation

$$d^2 = R^2 - 2Rr$$

(d being the distance between the incenter and circumcenter) is exactly a statement that the Jacobian of (R, r, d) has rank at most 2 everywhere, collapsing three ostensibly independent quantities onto a single surface. This is the precise sense in which searching for relations among exactly three derived scalars -- and not four -- is the search for something structurally new.

4. Detecting rank-deficiency with a bottleneck autoencoder

Given a candidate triple (f_1, f_2, f_3) , we want a numerical test for whether the induced map F has full rank 3 generically (data fills a 3D region) or is everywhere rank at most 2 (data confined to a 2D surface). We sample many random triangles and evaluate (f_1, f_2, f_3) on each, producing a point cloud X in $\mathbb{R}^3$.

An autoencoder with a 2-dimensional latent bottleneck -- an encoder $e: \mathbb{R}^3 \rightarrow \mathbb{R}^2$ and a decoder $d: \mathbb{R}^2 \rightarrow \mathbb{R}^3$, trained to minimize the expected squared reconstruction error -- is exactly searching for the best rank-2, nonlinear approximation to the data:

- If the data truly lies on a 2D surface (an exact relation holds), a 2-dimensional latent code is not actually a bottleneck for it -- a sufficiently expressive encoder/decoder can drive reconstruction error to near zero: the encoder learns a local chart of the surface, and the decoder learns its inverse.
- If the data genuinely fills an open 3D region, no continuous map factoring through a 2D intermediate space can reconstruct it without loss: an entire dimension of variation is necessarily discarded, so reconstruction error is bounded away from zero, on the order of the data's own extent in the discarded direction.

This gives a clean signature: near-zero reconstruction error with a 2-dimensional bottleneck is evidence for a relation among the three chosen quantities; error comparable to the data's own scale is consistent with the generic, full-rank, no-relation case.

5. Calibrating “near zero” with a permutation null

An absolute error threshold is not meaningful across different triples: they differ in units, scale, and marginal shape, and since all derived scalars are ultimately functions of the same three triangle parameters, some triples are mildly correlated even without an exact relation. We need a triple-specific baseline for “no relation.”

We construct a null sample X' by independently permuting each of the three columns of X . This preserves each quantity's own marginal distribution while completely destroying the joint dependency between them. Any real functional relation is annihilated by this shuffle. Training the same autoencoder on X' (repeated a few times) estimates the error expected for three quantities with these marginals but no shared structure. Comparing the real error against this null's mean and standard deviation, via a z-score or their ratio, yields a self-calibrated significance measure that is robust to units and distribution shape. A small ratio is the fingerprint of an unexpected relation.

6. From detection to an explicit formula

The autoencoder step is a screening tool: it flags which triples are suspicious, not what the relation says. Once a triple is flagged, we build a dictionary of monomials in (f_1, f_2, f_3) up to a chosen degree D , evaluate them on the sampled data to form a design matrix Φ (rescaling each column to comparable variance), and look for a linear combination of its columns that vanishes on every sample -- the right singular vector belonging to the smallest singular value of Φ .

Applied to (R, r, d) at degree 2, this recovers -- up to overall scale and sign, exactly the ambiguity inherent in reading off a null vector -- Euler's relation on every sampled triangle, found purely by sampling random triangles and asking a null-space question, with no prior knowledge of the formula. A very small trailing singular value (relative to the largest) is itself independent confirmation that an exact low-degree polynomial relation exists, complementary to the autoencoder's nonlinear, degree-agnostic detector.

7. The full pipeline

- Enumerate candidate triples of derived scalar quantities.
- For each triple, sample random triangles once, train a 2-bottleneck autoencoder, and compare its held-out reconstruction error to a column-shuffled null (several repeats).
- Rank triples by how much better the real data compresses than its null counterpart (a small ratio indicates a strong candidate).
- For top candidates, fit a low-degree polynomial null-space relation and validate that it holds (residual near zero) on fresh samples.

The `triangle_relations.discovery.verify_euler_relation` module runs this exact pipeline on (R, r, d) as a worked example, confirming it rediscovers Euler's relation end to end.